Run Nginx in Docker on AWS EC2

A step-by-step runbook that takes you from a new cloud server to your own Docker image serving a website in a container. Every command is explained, every output is shown, and every failure has a fix.

Project 2 of 3 | AWS EC2 + Docker + Nginx | About 45 minutes | Level: beginner



Know what you built. Unlike Project 1, here Nginx really runs: inside a Docker container, on a server you control. The roadmap infographic shows a local machine at localhost:8080. This project ran on an AWS EC2 server instead, so you reach it through the server's public address.

My implementation

Facts confirmed from my own screenshots. Figures 1 to 6 below are the evidence.

Server

AWS EC2 in Europe (Stockholm). Amazon Linux 2023, t3.micro, 8 GiB storage.

Docker

Installed with yum (docker 25.0.18 packages), started and enabled with systemctl.

Image

Container

`nginx-custom:v1`, 163 MB, built from the official `nginx:latest` image.

`nginx-web`, running detached, port mapping 8080:80.

Result

The NginxVerse page loads in a browser at the server’s public address on port 8080.

Before you start

You need an AWS account and a way to open a terminal on the server.

You need	Why	Check it
AWS account	Hosts the EC2 server	Sign in to the AWS console
Key pair (.pem file)	Lets you connect to the server with SSH	Created during launch in step 1
A terminal or browser	Browser-based EC2 Instance Connect or an SSH client	Either works
A web browser	Verifies the site at the end	Any current browser

Cost. An EC2 server is billed while it is running. Step 13 shows how to stop it when you finish.

The thirteen steps

Follow them in order. Each step ends with a check, so you know it worked before you continue.

1 Launch the EC2 server

Docker needs a Linux machine to run on. In this project that machine is an AWS EC2 virtual server.

1. In the AWS console open **EC2**, then **Instances**, then **Launch instances**.
2. **Name:** `ec2-user`, as in Figure 1.
3. **Application and OS Image:** Quick Start, **Amazon Linux** (Amazon Linux 2023 AMI).
4. **Instance type:** `t3.micro`.
5. **Key pair:** create or choose one and keep the downloaded `.pem` file safe. You need it for SSH.
6. **Network settings:** create a **new security group**.
7. **Storage:** 1 volume of 8 GiB. **Number of instances:** 1.
8. Click **Launch instance**.

An **AMI** is the operating system image the server starts from. A **security group** is a virtual firewall around the server. Figure 1 is the launch form before launching.

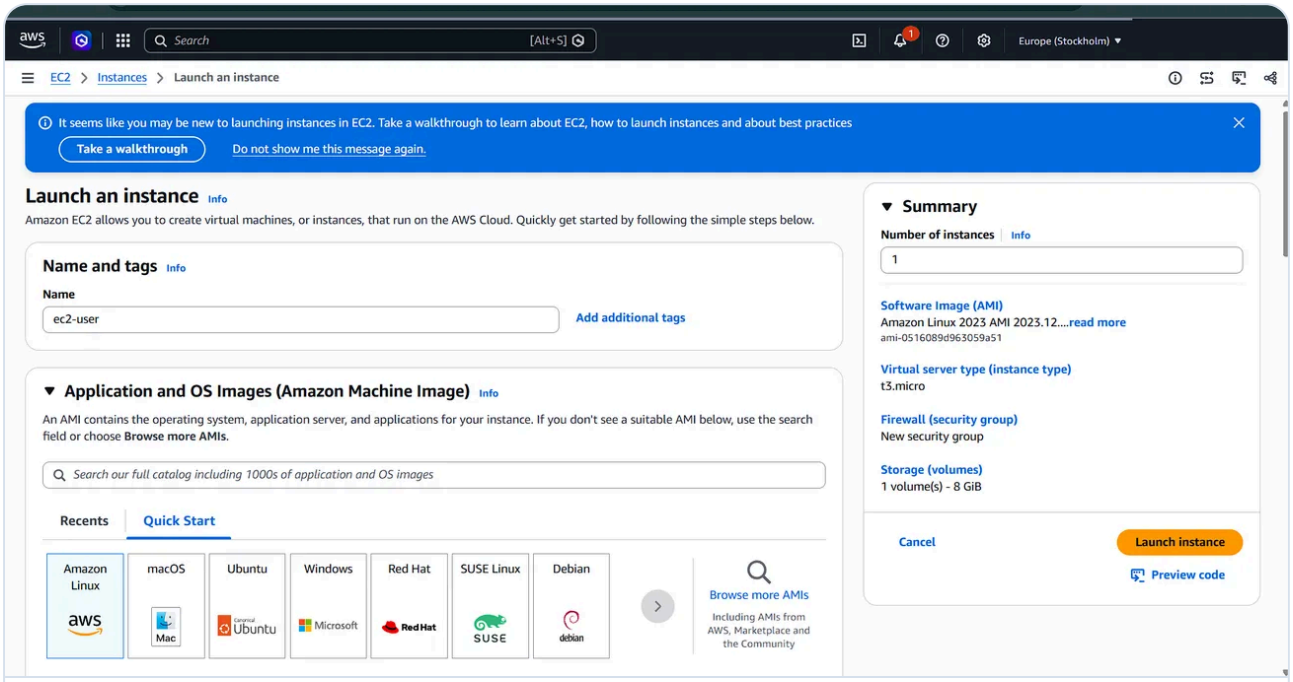


Figure 1: EC2 launch form (Europe, Stockholm)

- Name is `ec2-user` and Quick Start has Amazon Linux selected.
- The Summary panel shows 1 instance, the Amazon Linux 2023 AMI, type `t3.micro`, a new security group and 1 volume of 8 GiB.
- The account ID and name in the top bar were hidden before publishing.

Success looks like

The Instances list shows your server with state **Running** and a public IPv4 address.

If it fails: Launch instance is disabled
A required field is missing, usually the key pair. Choose or create one.

2 Open port 8080 in the security group

AWS blocks inbound traffic unless a rule allows it. The container will listen on 8080, so the browser cannot reach it until you add a rule.

1. Select your instance, then open the **Security** tab.
2. Click the security group link, then **Edit inbound rules**.
3. Click **Add rule**. Type: **Custom TCP**. Port range: `8080`.
4. Source: **My IP** for testing, or **Anywhere-IPv4** (`0.0.0.0/0`) for a public demo.
5. Click **Save rules**. Keep the SSH rule on port 22 that launch created.

None of the screenshots show this screen, but the browser in Figure 6 reached port 8080 on the server, which requires a rule like this.

Success looks like

The inbound rules list includes Custom TCP on port 8080.

If it fails: Browser times out later
The rule is missing, or the source is a different IP from the one you are using now.

3 Connect to the server

All the Docker work happens in a terminal on the server.

Option A: in the browser

Select the instance, click **Connect**, choose **EC2 Instance Connect**, then **Connect**.

Option B: SSH from your computer

your computer Copy

```
chmod 400 your-key.pem
ssh -i your-key.pem ec2-user@YOUR-EC2-PUBLIC-IP
```

`chmod 400` makes the key readable only by you, which SSH requires. `-i` names the key file. `ec2-user` is the default user on Amazon Linux. Replace `YOUR-EC2-PUBLIC-IP` with the address in the console.

Switch to a root shell

server Copy

```
sudo su
```

The prompts in the screenshots look like `[root@ip-172-31-23-195 ec2-user]#`: the `#` means a root shell. The commands below are shown exactly as in the screenshots, without `sudo`. If you stay as `ec2-user`, put `sudo` in front of each one.

Success looks like

Your prompt appears and `whoami` prints your user name.

> **If it fails: Permission denied (publickey)**
> Wrong key file or wrong user name. Use the key chosen at launch and the user `ec2-user`.

4 Install Docker

Docker is the program that builds images and runs containers. It is not installed on a new server.

server (root shell) Copy

```
yum update -y
yum install docker -y
```

`yum` is the Amazon Linux package manager. `update` refreshes installed packages. `install docker` adds the Docker engine. `-y` answers “yes” to confirmation prompts so the command does not pause.

expected output (end of install)

```
Installed:
  docker-25.0.18-1.amzn2023.0.2.x86_64 ...
Complete!
```

Check your work

server Copy

```
docker --version
```

Success looks like

The install ends with **Complete!** and the version command prints a Docker version (see Figure 2 in the next step).

> **If it fails: yum: command not found**
> You are not on Amazon Linux. This guide uses Amazon Linux 2023.

5 Start Docker and enable it at boot

Installing does not start the service. Enabling makes it start again after every reboot.

server (root shell) Copy

```
systemctl start docker
systemctl enable docker
docker --version
systemctl status docker
```

`systemctl` controls services. `start` runs Docker now. `enable` creates a link so it starts at boot, and prints the symlink message you can see in Figure 2. `status` reports whether it is running.

expected output of status (illustrative)

```
● docker.service - Docker Application Container Engine
     Active: active (running)
```



Figure 2: Docker installation and startup

- The install list includes `docker-25.0.18`, `containerd-2.3.7` and `runc-1.4.3`, followed by **Complete!**.
- `systemctl start docker` and `systemctl enable docker` run next, and the symlink message confirms the service was enabled.
- `docker --version` is then run.

Success looks like

`systemctl status docker` shows **active (running)**.

> **If it fails: Unit docker.service not found**
> The install did not finish. Run `yum install docker -y` again.

6 Create the project folder

A dedicated folder keeps the Dockerfile and website files together. Docker sends this folder to the builder as the build context.

server Copy

```
mkdir nginx-docker-project
cd nginx-docker-project
```

`mkdir` creates the folder. `cd` moves into it, so every later command runs there.

Success looks like

The prompt now ends with `nginx-docker-project]#`, as in Figures 3 and 4.

If it fails: File exists

The folder is already there. Just run `cd nginx-docker-project`.

7 Create index.html

This is the page Nginx will serve instead of its default welcome page.

server Copy

```
vi index.html
```

`vi` is a text editor available on every server. It has two modes:

Key	What it does
`i`	Insert mode, so you can type
`Esc`	Leave insert mode
`:wq` then Enter	Save and quit
`:q!` then Enter	Quit without saving

Minimal page to follow along (the NginxVerse page in Figure 6 is the author's own, larger page)

index.html Copy

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>NginxVerse</title>
</head>
<body>
  <h1>Nginx running in Docker on AWS EC2</h1>
</body>
</html>
```

Check your work

server Copy

```
ls
```

Success looks like

`ls` lists `index.html`.

If it fails: Changes not saved

You were still in insert mode. Press Esc, then type `:wq` and Enter.

8 Write the Dockerfile

The Dockerfile is the recipe Docker follows to build your image.

server Copy

```
vi Dockerfile
```

Dockerfile (the two instructions visible in the build log in Figure 3)

Dockerfile Copy

```
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
```

`FROM nginx:latest` starts from the official Nginx image, which already contains Nginx. `COPY index.html /usr/share/nginx/html/index.html` copies your page into the folder Nginx serves, replacing the default page. The file name must be exactly `Dockerfile`, with a capital D and no extension.

Check your work

server Copy

```
ls
```

expected output

```
Dockerfile  index.html
```

Success looks like

Both `Dockerfile` and `index.html` are listed.

If it fails: Build later says the Dockerfile cannot be found

The file is misnamed (for example dockerfile.txt). Rename it to `Dockerfile`.

9 Build the Docker image

Building turns the Dockerfile and your files into an image, a packaged and reusable copy of the Nginx server with your page inside.

server, inside nginx-docker-project Copy

```
docker build -t nginx-custom:v1 .
```

`docker build` builds an image. `-t nginx-custom:v1` names (tags) it: the name is `nginx-custom` and the version tag is `v1`. The final `.` means “use the current folder as the build context”. Do not leave it out.

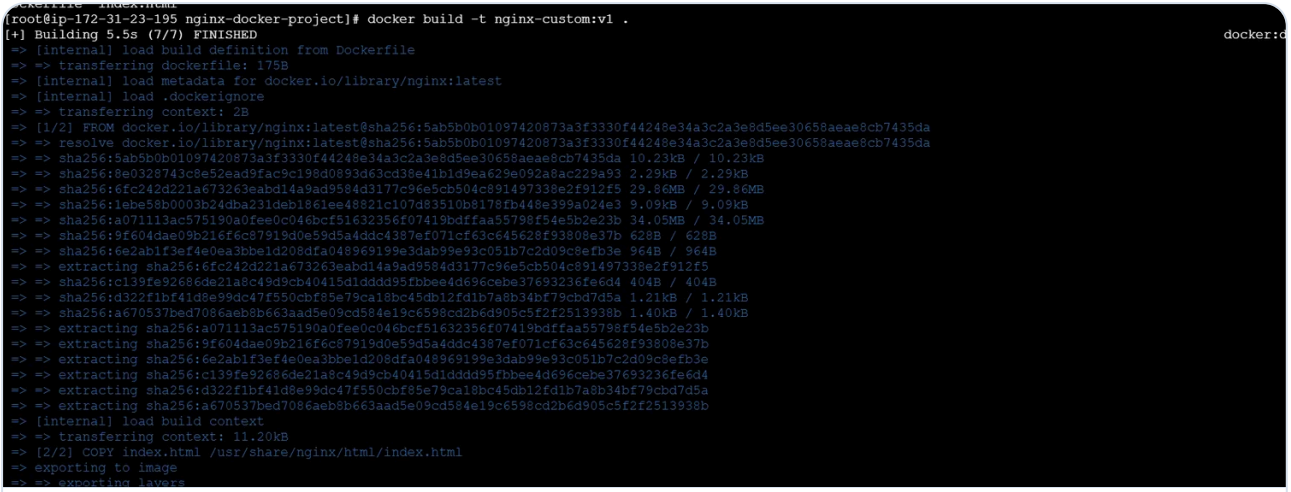


Figure 3: Output of docker build

- **Building 5.5s (7/7) FINISHED** means all seven build steps completed.
- Docker loads the 175-byte Dockerfile and reads the base image `docker.io/library/nginx:latest`.
- The base image layers are downloaded, then extracted.
- `[2/2] COPY index.html /usr/share/nginx/html/index.html` is the step that adds your page.
- The last lines export the layers and name the image `nginx-custom:v1`.

Success looks like

The log ends with the image being exported and named, with no error lines.

If it fails: COPY failed: file not found

`index.html` is not in the folder you built from. Run `ls` and build from the right folder.

10 Check the image exists

Confirms the build produced an image before you try to run it.

server Copy

```
docker images
```

Lists images stored on this server. Columns: repository, tag, image ID, age and size.

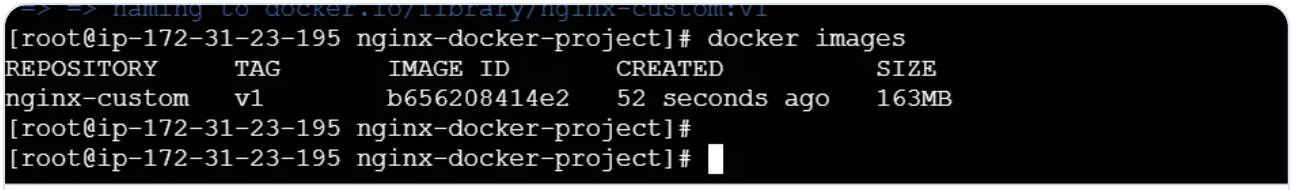


Figure 4: docker images after the build

- Repository `nginx-custom`, tag `v1`.
- Image ID b656208414e2, created 52 seconds ago.
- Size is 163MB, mostly the Nginx base image.

Success looks like

`nginx-custom` with tag `v1` is listed.

If it fails: Image is missing

The build failed. Scroll up in the terminal and read the first error line.

11 Run the container

An image is only a template. Running it creates a live container that serves the website.

server Copy

```
docker run -d --name nginx-web -p 8080:80 nginx-custom:v1
docker ps
```

Part	Meaning
`docker run`	Create and start a container
`-d`	Detached: run in the background and give the prompt back
`--name nginx-web`	Name the container so you can manage it by name
`-p 8080:80`	Port mapping: server port **8080** forwards to container port **80**, where Nginx listens
`nginx-custom:v1`	The image to start from

`docker ps` lists running containers.

expected output of docker ps (illustrative)

```
CONTAINER ID   IMAGE             COMMAND                  STATUS         PORT
3f2a9c1d7e44   nginx-custom:v1   "/docker-entrypoint.…"   Up 9 seconds   0.0.
```

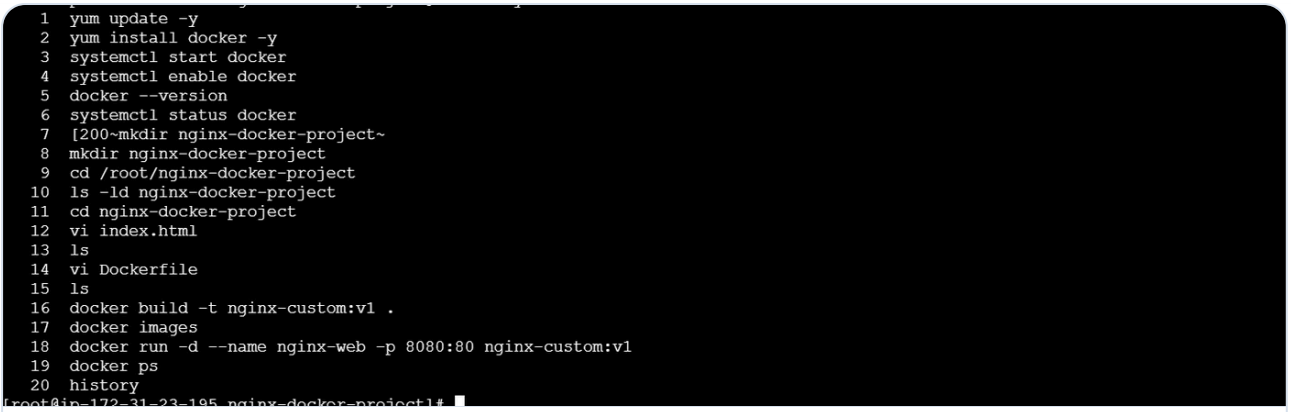


Figure 5: Command history for the whole project

- Lines 1 to 6 install, start and check Docker.
- Lines 8 to 15 create the folder and the two files with `vi`.
- Lines 16 and 17 build and list the image. Line 18 runs the container with `-p 8080:80`, and line 19 is `docker ps`.
- Line 7 contains stray characters from pasting into the terminal. The clean command follows on line 8.

Success looks like

`docker ps` shows `nginx-web` with status **Up** and ports `0.0.0.0:8080->80/tcp`.

> **If it fails: port is already allocated**
> Something already uses 8080. Run `docker ps`, stop that container, or use `-p 8081:80` and open port 8081 in the security group.

12 Verify in the browser

Proves the whole chain works: firewall, server, port mapping, container and Nginx.

Open `http://YOUR-EC2-PUBLIC-IP:8080` in a browser. Use **http**, not https. Find the address in the EC2 console under **Public IPv4 address**.

Optional check from the server itself

server Copy

```
curl -I http://localhost:8080
```

expected output (illustrative)

```
HTTP/1.1 200 OK
Server: nginx/1.x
Content-Type: text/html
```

`-I` asks for headers only. `200` means success, and the `Server: nginx` header shows Nginx answered.

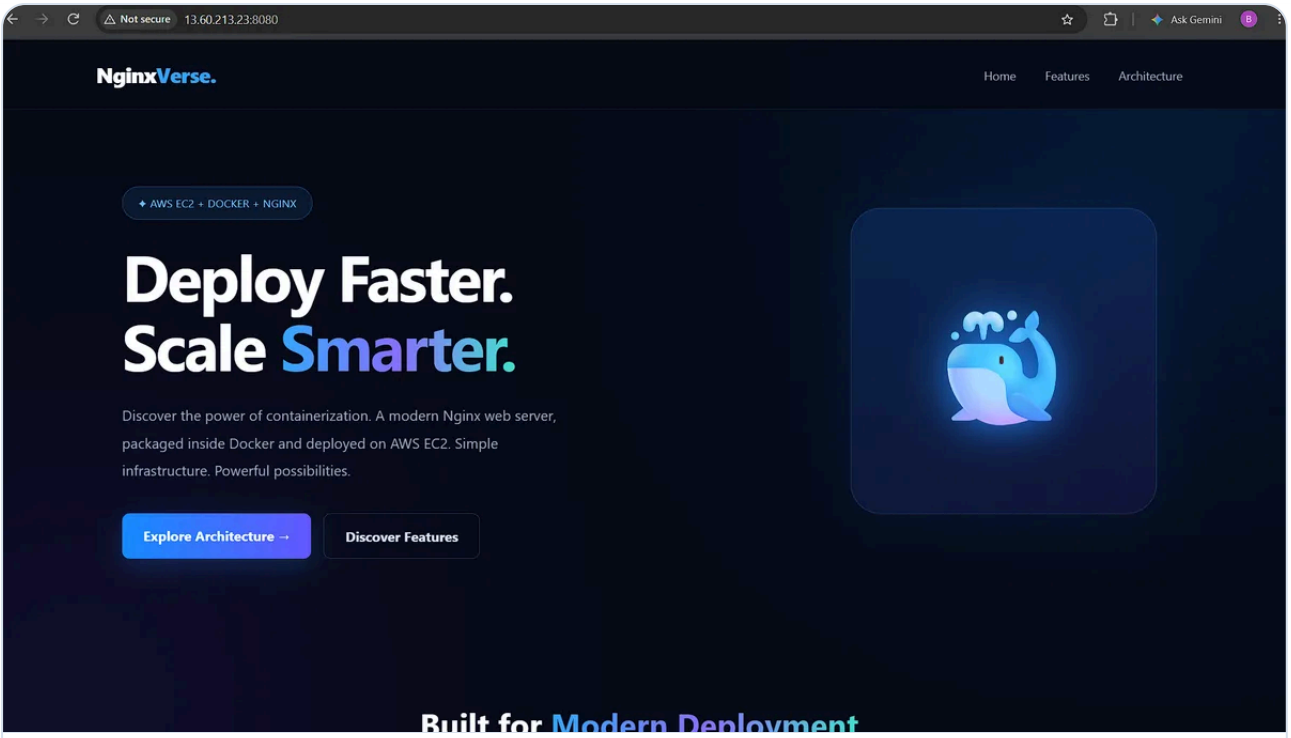


Figure 6: The deployed site in a browser

- The address bar shows the EC2 public address on port 8080. Chrome labels it “Not secure” because the connection is plain HTTP.
- The page shows the NginxVerse logo, the badge “AWS EC2 + DOCKER + NGINX” and the heading “Deploy Faster. Scale Smarter.”.
- The Docker whale graphic and the “Explore Architecture” and “Discover Features” buttons are visible.

Success looks like

Your page loads, and `curl -I` returns **200** from the server.

> **If it fails: Page does not load**
> Check the security group rule (step 2), that `docker ps` shows the container, and that you used http and the current public address.

13 Inspect, stop and clean up

Stopping resources avoids charges and leaves the server tidy. These commands are standard practice and are not part of the author's command history.

server Copy

```
docker logs nginx-web
docker stop nginx-web
docker rm nginx-web
docker rmi nginx-custom:v1
```

`logs` shows Nginx's output. `stop` halts the container. `rm` deletes it (`-f` not needed once stopped). `rmi` deletes the image. Safety: removing an image or container cannot be undone, but you can rebuild it from the Dockerfile.

Then in the AWS console

Select the instance, then **Instance state**, then **Stop instance** to pause billing for compute, or **Terminate instance** to delete it permanently.

Success looks like

`docker ps -a` no longer lists nginx-web, and the instance state is Stopped or Terminated.

> **If it fails: Image is being used by a container**
> Remove the container first with `docker rm nginx-web`.

When something goes wrong

These are common problems for this setup. Find the symptom, run the check, apply the fix.

Symptom	Likely cause	Check	Fix
Browser times out on port 8080	Security group has no inbound rule for 8080, or	EC2, Security, Inbound rules	Add or correct the Custom TCP 8080

	the source is not your current IP		rule
Page does not load over https	The container serves plain HTTP only	Address bar	Use `http://`, not https
Address stopped working after a restart	The public IP changes when an instance is stopped and started	EC2 console, public IPv4 address	Use the new address, or attach an Elastic IP
`Cannot connect to the Docker daemon`	Docker is not running	`systemctl status docker`	`systemctl start docker`
`permission denied on the Docker socket`	Running as a normal user without access	`whoami`	Use `sudo`, or add the user to the `docker` group
`COPY failed: file not found`	`index.html` is not in the build folder	`ls`	Run the build from the folder that contains both files
`port is already allocated`	Another process or container uses 8080	`docker ps`	Stop that container, or map another port such as 8081:80
`container name is already in use`	A container named nginx-web already exists	`docker ps -a`	`docker rm nginx-web`, then run again
Old page still showing after a change	The image was not rebuilt, or the old container is still running	`docker images`, `docker ps`	Rebuild, remove the old container, run a new one

Security notes

Avoid working as root

The screenshots show commands run from a root shell. That is convenient for

Limit the open port

Allowing port 8080 from anywhere makes the site public. For testing, set

learning, but a mistake as root can damage the server. In real work, use `ec2-user` with `sudo`.

the source to your own IP. Open it to everyone only for a demo.

Plain HTTP

The site is served without encryption, which is why the browser says “Not secure”. Do not put logins or private data on it.

Protect your key and account

Never share the .pem key. Hide the account ID and name before posting AWS screenshots.

Pin image versions

The build used `nginx:latest`, which changes over time. For repeatable builds, pin a specific version tag.

Stop paying

Stop or terminate the instance when finished, and remove unused containers and images.

What you can say you learned

Docker

- Writing a Dockerfile
- Building and tagging an image
- Running a container with port mapping

AWS EC2

- Launching a Linux server
- Security groups as a firewall
- Reaching a service by public address

Linux administration

- Installing packages with yum
- Managing services with systemctl
- Editing files with vi

Nginx

- Where Nginx serves files from
- Replacing the default page

Final checklist

Tick each item as you finish it. Your progress is saved in this browser.

0 of 14 complete

- [ ] EC2 instance launched and running
- [ ] Port 8080 allowed in the security group
- [ ] Connected to the server
- [ ] Docker installed
- [ ] Docker started and enabled
- [ ] Project folder created
- [ ] index.html created
- [ ] Dockerfile created
- [ ] Image built as nginx-custom:v1
- [ ] Image appears in docker images
- [ ] Container nginx-web running with 8080:80
- [ ] Site loads in the browser on port 8080
- [ ] Logs checked and containers cleaned up
- [ ] Instance stopped or terminated

Official references

Dockerfile reference docker build docker run Official Nginx image

Get started with Amazon EC2